

1.) FCFS

```
package fcfssimple;

import java.util.Scanner;

public class FCFSsimple {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of processes: ");

        int n = sc.nextInt();

        int[] bt = new int[n];

        int[] wt = new int[n];

        int[] tat = new int[n];

        for (int i = 0; i < n; i++) {

            System.out.print("Enter Burst Time for Process " + (i + 1) + ": ");

            bt[i] = sc.nextInt();

        }

        wt[0] = 0;

        for (int i = 1; i < n; i++) {

            wt[i] = wt[i - 1] + bt[i - 1];

        }

        for (int i = 0; i < n; i++) {

            tat[i] = wt[i] + bt[i];

        }

        System.out.println("\nPID\tBT\tWT\tTAT");

        for (int i = 0; i < n; i++) {

            System.out.println((i + 1) + "\t" + bt[i] + "\t" + wt[i] + "\t" + tat[i]);

        }

    }

}
```

Output:

run:

Enter number of processes: 3

Enter Burst Time for Process 1: 6

Enter Burst Time for Process 2: 4

Enter Burst Time for Process 3: 8

PID	BT	WT	TAT
1	6	0	6
2	4	6	10
3	8	10	18

BUILD SUCCESSFUL (total time: 5 seconds)

2.) System Call:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if(pid < 0) {
```

```
        printf("Fork Failed\n");
```

```
    }
```

```
    else if(pid == 0) {
```

```
        printf("Child Process:\n");
```

```
        printf("Child PID = %d\n", getpid());
```

```
        printf("Parent PID = %d\n", getppid());
```

```
    }
```

```
    else {
```

```
        printf("Parent Process:\n");
```

```
        printf("Parent PID = %d\n", getpid());
```

```
        printf("Child PID = %d\n", pid);
```

```

    }
    return 0;
}

```

Output:

Parent Process:

Parent PID=2456

Child PID=2457

Child Process:

Child PID=2457

Parent PID=2456

3.) Page Replacement Algorithm FCFS:

```

package replacement.algorithm.fcfs;
import java.util.*;
public class ReplacementAlgorithmFCFS {
    public static void main(String[] args) {
        int[] pages = {1, 3, 0, 3, 5, 6};
        int frames = 3;
        Queue<Integer> memory = new LinkedList<>();
        Set<Integer> pageSet = new HashSet<>();
        int pageFaults = 0;
        System.out.println("FCFS (FIFO) Page Replacement Algorithm\n");
        for (int page : pages) {
            if (!pageSet.contains(page)) {
                pageFaults++;
                if (memory.size() == frames) {
                    int removedPage = memory.poll();
                    pageSet.remove(removedPage);
                }
            }
        }
    }
}

```

```

        memory.add(page);
        pageSet.add(page);
    }
    System.out.println("Page: " + page + " -> Memory: " + memory);
}
System.out.println("\nTotal Page Faults = " + pageFaults);
}
}

```

Output:

FCFS (FIFO) Page Replacement Algorithm

Page: 1 -> Memory: [1]

Page: 3 -> Memory: [1, 3]

Page: 0 -> Memory: [1, 3, 0]

Page: 3 -> Memory: [1, 3, 0]

Page: 5 -> Memory: [3, 0, 5]

Page: 6 -> Memory: [0, 5, 6]

Total Page Faults = 5

BUILD SUCCESSFUL (total time: 0 seconds)

4.) LRU

```

package lru;
import java.util.*;
public class LRU {
    public static void main(String[] args) {
        int[] pages = {1, 3, 0, 3, 5, 6};
        int frames = 3;
        Set<Integer> memory = new LinkedHashSet<>();
        int faults = 0;
        for (int page : pages) {

```

```

        if (!memory.contains(page)) {
            faults++;
            if (memory.size() == frames) {
                int lru = memory.iterator().next();
                memory.remove(lru);
            }
        } else {
            memory.remove(page);
        }
        memory.add(page);
        System.out.println("Page: " + page + " Memory: " + memory);
    }
    System.out.println("Total Page Faults = " + faults);
}
}

```

Output:

Page: 1 Memory: [1]

Page: 3 Memory: [1, 3]

Page: 0 Memory: [1, 3, 0]

Page: 3 Memory: [1, 0, 3]

Page: 5 Memory: [0, 3, 5]

Page: 6 Memory: [3, 5, 6]

Total Page Faults = 5

BUILD SUCCESSFUL (total time: 0 seconds)

5.) Clock Synchronization using Berkely Algorithm:

```
package clocksyncsimple;
```

```
public class ClockSyncSimple {
```

```
    public static void main(String[] args) {
```

```

int[] clocks = {100, 95, 105, 98};

System.out.println("Before Synchronization:");
for (int i = 0; i < clocks.length; i++) {
    System.out.println("Node " + i + " Time: " + clocks[i]);
}

int masterTime = clocks[0];

int sumDifference = 0;
for (int i = 0; i < clocks.length; i++) {
    sumDifference += (clocks[i] - masterTime);
}

int averageDifference = sumDifference / clocks.length;

int newMasterTime = masterTime + averageDifference;

for (int i = 0; i < clocks.length; i++) {
    int adjustment = newMasterTime - clocks[i];
    clocks[i] += adjustment;
}

System.out.println("\nAfter Synchronization:");
for (int i = 0; i < clocks.length; i++) {
    System.out.println("Node " + i + " Time: " + clocks[i]);
}
}

```

Output:

Before Synchronization:

Node 0 Time: 100

Node 1 Time: 95

Node 2 Time: 105

Node 3 Time: 98

After Synchronization:

Node 0 Time: 100

Node 1 Time: 100

Node 2 Time: 100

Node 3 Time: 100

BUILD SUCCESSFUL (total time: 0 seconds)

6.) Concurrency control algorithms – Semaphore:

```
package sharedresourcesemaphore;

import java.util.concurrent.Semaphore;

public class SharedResourceSemaphore {

    static Semaphore semaphore = new Semaphore(1);

    static class Task extends Thread {

        Task(String name) {

            super(name);

        }

        public void run() {

            try {

                semaphore.acquire();

                System.out.println(getName() + " entered critical section");

                Thread.sleep(500);

                System.out.println(getName() + " leaving critical section");

                semaphore.release();

            } catch (InterruptedException e) {

                e.printStackTrace();

            }

        }

    }

}
```

```

    }

    public static void main(String[] args) {

        new Task("Thread-1").start();
        new Task("Thread-2").start();
    }
}

```

Output:

```

Thread-1 entered critical section
Thread-1 leaving critical section
Thread-2 entered critical section
Thread-2 leaving critical section
BUILD SUCCESSFUL (total time: 1 second)

```

7.) Concurrency control algorithms - Mutual exclusion:

```

package sharedresource;

class SharedResource {
    public synchronized void use() {
        System.out.println(Thread.currentThread().getName() + " entered");
        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println(Thread.currentThread().getName() + " exited");
    }

    public static void main(String[] args) {
        SharedResource resource = new SharedResource();
        Runnable task = () -> resource.use();
        Thread t1 = new Thread(task, "Thread-1");
    }
}

```



```

Thread t2 = new Thread(task, "Thread-2");

t1.start();

t2.start();

try {

    t1.join();

    t2.join();

} catch (InterruptedException e) {

    Thread.currentThread().interrupt();

}

}

}

```

Output:

Thread-1 entered

Thread-1 exited

Thread-2 entered

Thread-2 exited

BUILD SUCCESSFUL (total time: 1 second)

8.) Implementation of Deadlock:

```

package deadlock;

class Deadlock {

    public static void main(String[] args) {

        Object A = new Object();

        Object B = new Object();

        Thread t1 = new Thread(() -> {

            synchronized (A) {

                System.out.println("T1 locked A");

                try { Thread.sleep(100); } catch (Exception e) {}

                synchronized (B) {

```

```

        System.out.println("T1 locked B");
    }
}
});

Thread t2 = new Thread(() -> {
    synchronized (B) {
        System.out.println("T2 locked B");
        try { Thread.sleep(100); } catch (Exception e) {}
        synchronized (A) {
            System.out.println("T2 locked A");
        }
    }
});

t1.start();
t2.start();
}
}

```

Output:

T1 locked A

T2 locked B

9.) Implementation of Distributed scheduling

package distributedschedulingsimple;

```

class Worker extends Thread {
    private int taskId;

    public Worker(int taskId) {
        this.taskId = taskId;
    }

    public void run() {
        System.out.println("Task " + taskId +

```

```

        " is executed by " + Thread.currentThread().getName());
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}

public class DistributedSchedulingSimple {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            Worker worker = new Worker(i);
            worker.start();
        }
        System.out.println("All tasks are assigned.");
    }
}

```

Output:

Task 1 is executed by Thread-0

All tasks are assigned.

Task 2 is executed by Thread-1

Task 4 is executed by Thread-3

Task 5 is executed by Thread-4

Task 3 is executed by Thread-2

BUILD SUCCESSFUL (total time: 1 second)